

Gestion scolaire - Projet CDA

Backend Spring Boot • Frontend React • PostgreSQL

Architecture

- • React + Vite
 - → Interface web, routage React Router, pages protégées, appels API via Axios
- • Spring Boot REST
 - → API REST exposée sous /api/**, contrôleurs par domaine métier, gestion centralisée des erreurs
- • JPA / Hibernate
 - → Persistance des entités, repositories Spring Data, schéma auto-géré (ddl-auto: update)
- • PostgreSQL
 - → Base relationnelle hébergée dans un conteneur Docker, image postgres:16-alpine
- • JWT
 - → Token signé HS256, durée 1 heure, envoyé automatiquement par Axios, routes frontend protégées
- • Docker
 - → docker-compose.yml lance PostgreSQL avec healthcheck et volume persistant

Démonstration

- 1. Connexion
 - → Saisir DOE / PasswOrd! → token JWT stocké, redirection automatique vers le dashboard
- 2. Élèves (CRUD + photo)
 - → Créer un élève, uploader une photo, voir la miniature apparaître immédiatement dans la liste, ouvrir la modification et voir l'aperçu via GET /api/elevés/{id}/photo
- 3. Notes / Bulletins
 - → Saisir une note (valeur entre 0 et 20), créer un bulletin (trimestre 1, 2 ou 3, format YYYY-YYYY)
- 4. Export PDF
 - → Cliquer sur "Télécharger PDF" → le backend génère un PDF-1.4 en Java pur, le navigateur propose le téléchargement

Points techniques

- • Architecture Controller/Service/Repository
 - → Séparation stricte : contrôleurs exposent les routes, services portent la logique, repositories accèdent aux données. Validations dans les DTO avec [jakarta.validation](#).
- • DTO + Mapper
 - → Chaque entité a un DTO dédié avec ses contraintes. Les Mapper font la conversion sans bibliothèque de mapping externe.
- • Upload photo
 - → Multipart [POST /api/eleves/{id}/photo](#), stockage dans [uploads/eleves/](#), chemin enregistré en base. Restitution via [GET /api/eleves/{id}/photo](#) avec détection MIME automatique.
- • PDF généré entièrement en Java natif
 - → Générateur 100 % maison en Java pur : construit le flux PDF-1.4 byte par byte, aucune bibliothèque externe (pas iText, pas OpenPDF).
- • Sécurité JWT
 - → JwtAuthenticationFilter lit le header Authorization, JwtService signe et valide le token HS256. Seul [/auth/login](#) est public, tout [/api/**](#) exige un token valide.

Conclusion

- ✓ Fonctionnalités principales terminées
 - → Authentification JWT, 10 CRUD, upload photo, export PDF, dashboard, Swagger documenté.
- ✓ Build, compile et lint validés
 - → mvn compile → BUILD SUCCESS, npm run build → 114 modules transformés, npm run lint → aucune erreur.